

PRACTICAL NO 06

Aim: Write a program to implement queue using linked lists.

Theory:

Write down about Queue

ALGORITHM:

Algorithm for Queue using Linked List

Step 1: Define Node Structure

- Each node has two parts:
 1. **data** → to store the value
 2. **next** → pointer to the next node

Maintain two pointers:

- **front** → points to the first node
 - **rear** → points to the last node
-

Step 2: Enqueue (Insert element at rear)

1. Create a new node with given value.
 2. If **rear == NULL** (queue is empty):
 - Set front = rear = newNode.
 3. Else:
 - Link rear->next = newNode.
 - Update rear = newNode.
 4. Print "value inserted".
-

Step 3: Dequeue (Remove element from front)

1. If **front == NULL**:
 - Queue is empty → print "Queue empty".
2. Else:
 - Store temp = front.

- Print "front->data removed".
 - Move front to front->next.
 - If front == NULL, set rear = NULL.
 - Free temp node.
-

Step 4: Display Queue

1. If **front == NULL** → print "Queue empty".
 2. Else:
 - Start from front.
 - Traverse nodes until NULL.
 - Print each node's data.
-

Step 5: Main Function

1. Call enqueue(10), enqueue(20), enqueue(30).
2. Call display() → shows all elements.
3. Call dequeue() → removes one element.
4. Call display() again.

Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
struct Node *front = NULL, *rear = NULL;
```

```
// Enqueue (insert)

void enqueue(int value) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;

    newNode->next = NULL;

    if (rear == NULL) { // if queue empty

        front = rear = newNode;

    } else {

        rear->next = newNode;

        rear = newNode;

    }

    printf("%d inserted\n", value);

}
```

```
// Dequeue (remove)

void dequeue() {

    if (front == NULL) {

        printf("Queue empty\n");

        return;

    }

    struct Node* temp = front;

    printf("%d removed\n", front->data);

    front = front->next;

    if (front == NULL) rear = NULL; // queue empty

    free(temp);

}
```

```
// Display

void display() {
```

```
struct Node* temp = front;
if (temp == NULL) {
    printf("Queue empty\n");
    return;
}
printf("Queue: ");
while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}
printf("\n");
}
```

```
// Main
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();

    dequeue();
    display();

    return 0;
}
```

OUTPUT:

10 inserted

20 inserted

30 inserted

Queue: 10 20 30

10 removed

Queue: 20 30